Git & GitHub
1주차 세미나

B!BimBab

Git의 기초와 구조, 설치 : 파일 이름을 더럽히지 않는 버전 관리와 3가지 영역

Contents

01. GIT 이란?

02. GITHUB란?

03. 버전 관리와 3가지 영역

04. GIT설치

01 GIT 이란?

01 GIT 이란 ?

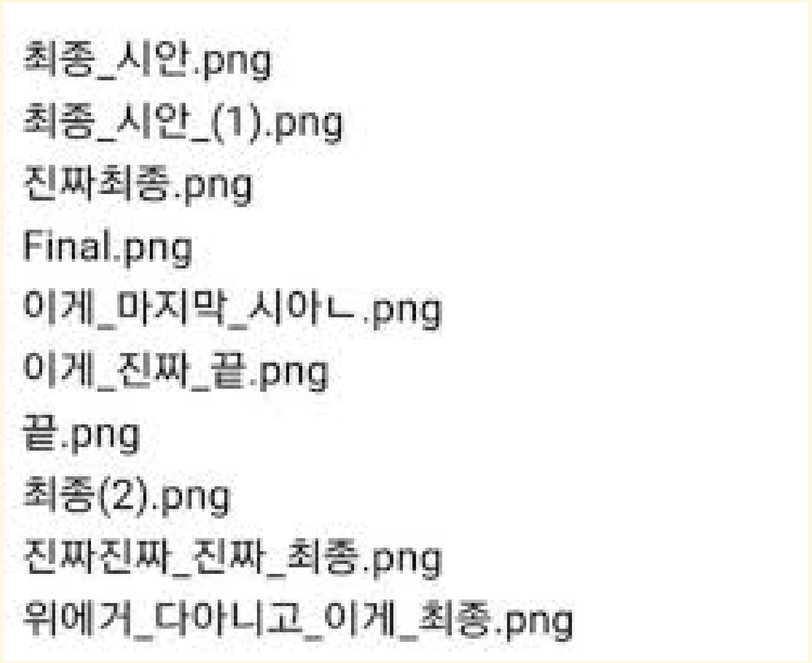
GIT : 버전 관리 시스템 (Version Control System) 의 일종

우리는 이미 파일의 이름을 바꿔가면서 파일 내용을 여러개의 버전으로 관리한 적이 있다.

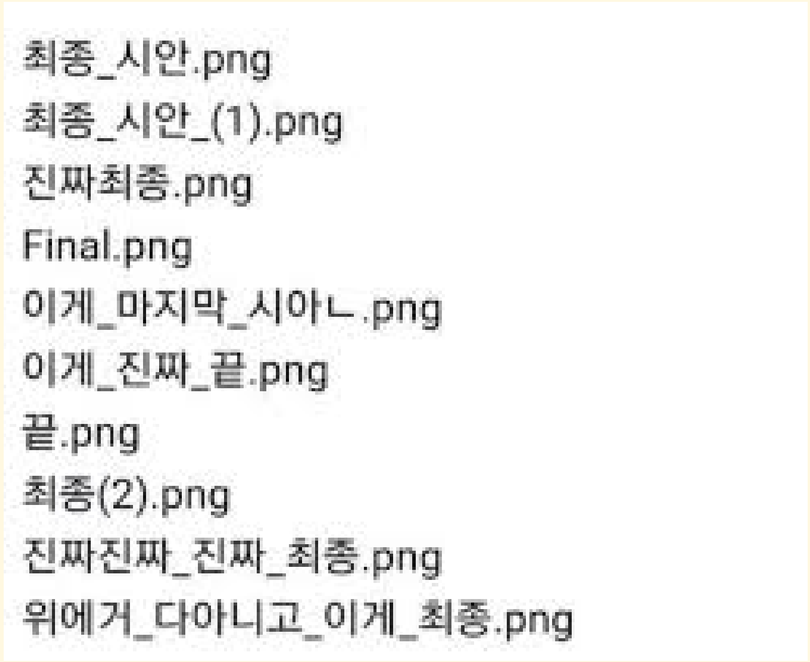
01 GIT 이란 ?

GIT : 버전 관리 시스템 (Version Control System) 의 일종

우리는 이미 파일의 이름을 바꿔가면서 파일 내용을 여러개의 버전으로 관리한 적이 있다.



01 GIT 이란?



문제 1. 어떤 버전에서 뭐가 바뀌었는지 알 수 없음

문제 2. 여러 파일이 쌓이면서 용량과 관리 부담이 커짐

문제 3. 예전 버전으로 되돌리고 싶을 때 수동으로 찾아야 함

01 GIT 이란 ?

파일의 이름은 늘 그대로 두고, 파일의 변경사항은 컴퓨터가 관리하게 해보자

Back up

Recovery

Collaboration

01 GIT 이란 ?

깃 근데 어렵던데요...?

간단한 프로젝트 + 깃의 복잡성 => 프로젝트의 복잡도 매우 올라갈지도..?

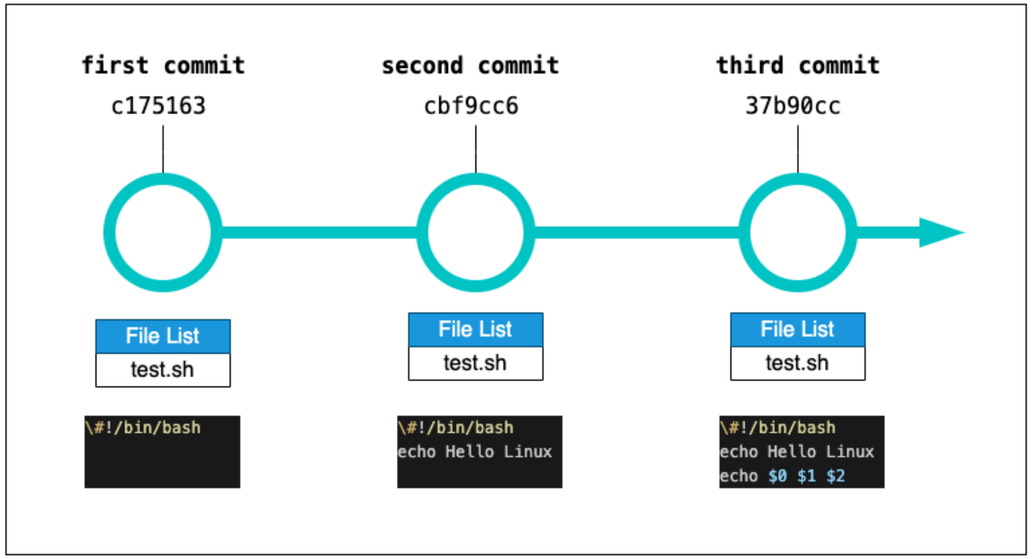
매우 복잡한 프로젝트 + 깃의 어느정도 복잡성 => 프로젝트의 복잡도가 어느정도 해결됨

01 GIT의 버전관리

스냅샷과 commit

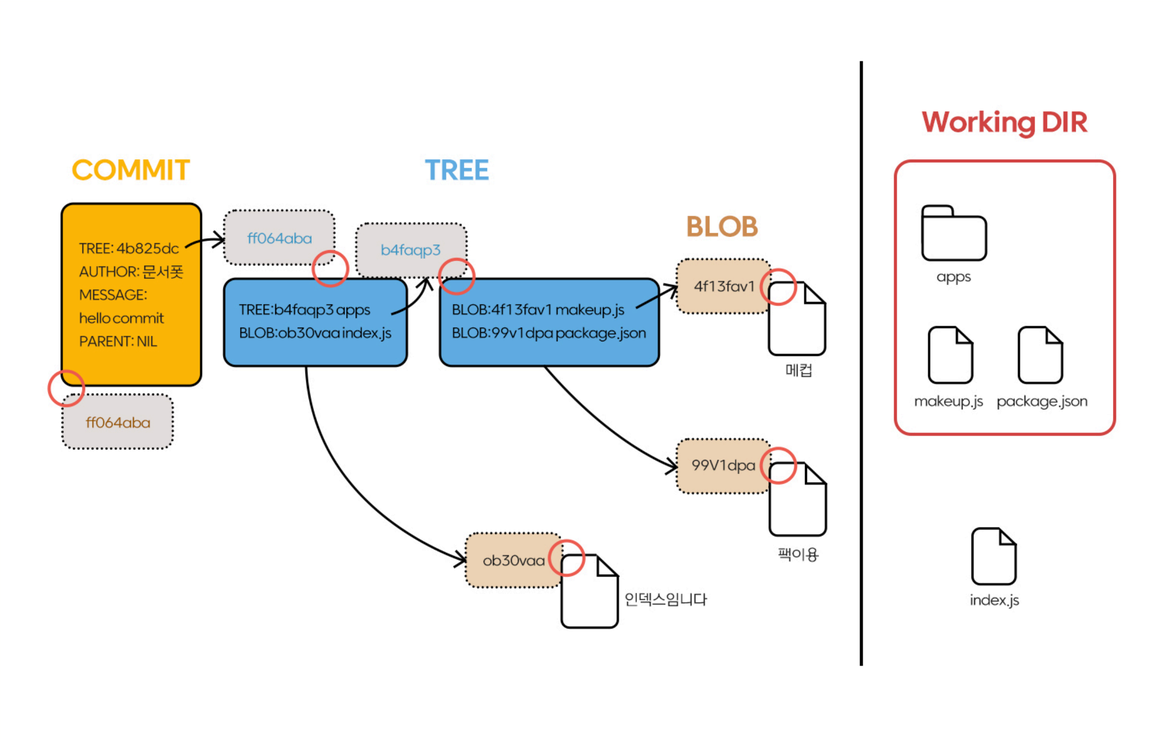
커밋은 git 의 버전관리의 기본 단위

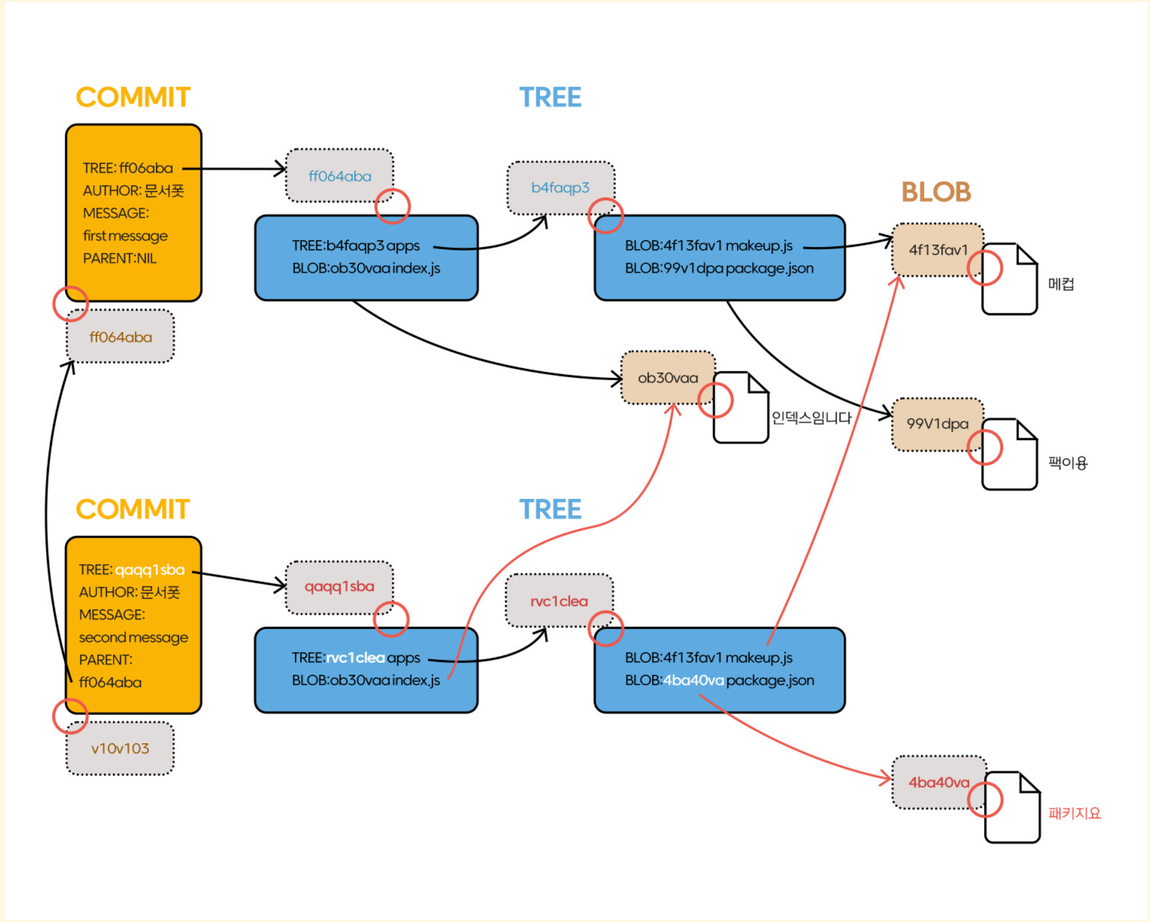
커밋 하나는 그 순간의 프로젝트의 전체를 찍은 사진 한 장(스냅샷!)!



01 GIT의 버전관리

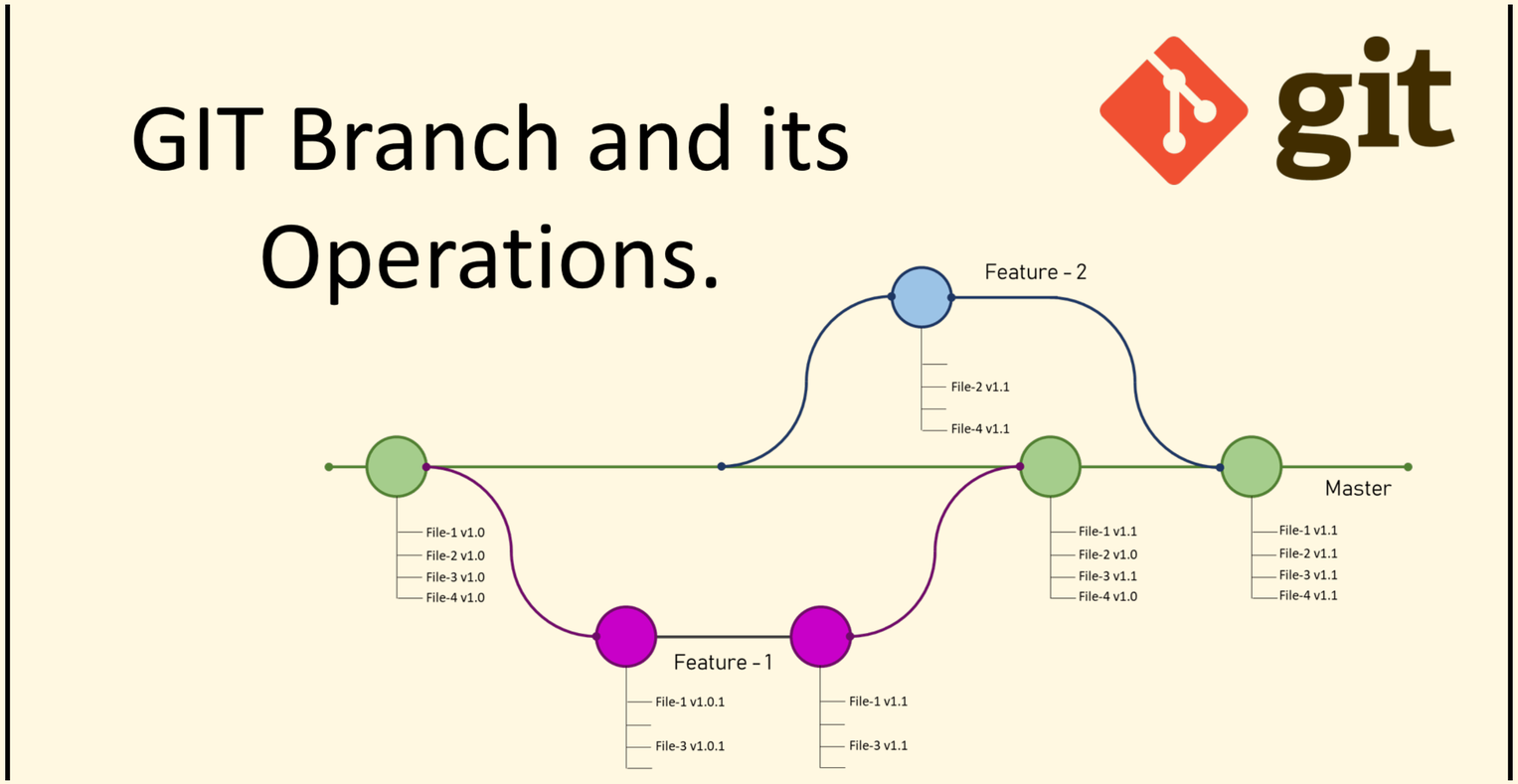
그럼 용량은?? → 논리적으로는 전체를 찍은 것처럼 보이지만, 물리적으로는 안 바뀐 부분은 포인터로 재활용한다





01 깃의 저장공간 branch

나뭇가지! main 줄기에서 갈라져 나와서 독립적으로 새 기능을 만드는 공간
브랜치는 그 브랜치의 가장 최신 커밋을 가리키고 있음.
새 기능을 개발할 때, 실험적인 시도를 할 때, 급한 버그를 고칠 때, 여러 명이 각자 다른 작업을 할 때 브랜치를 파서 작업한다.



02

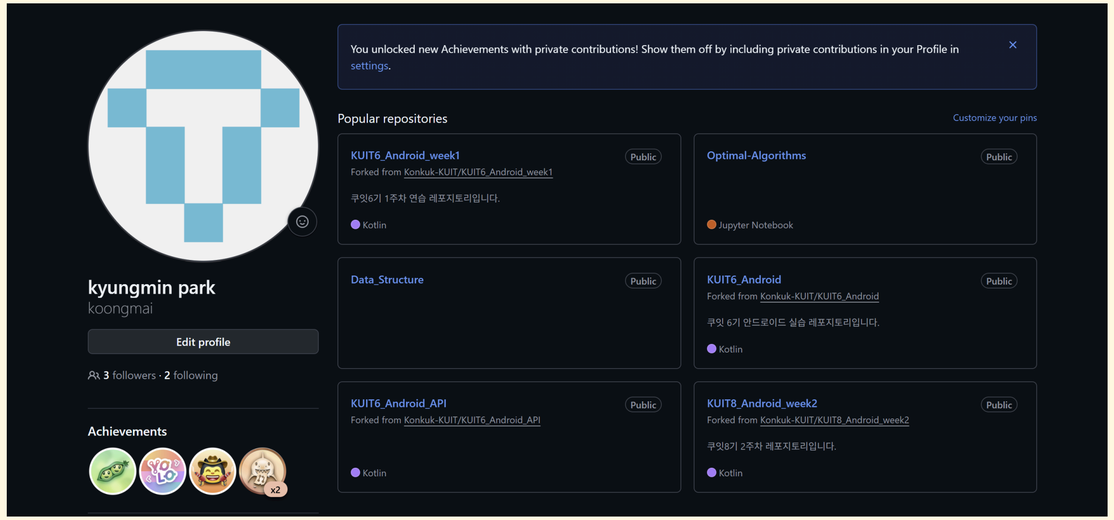
GITHUB 란?

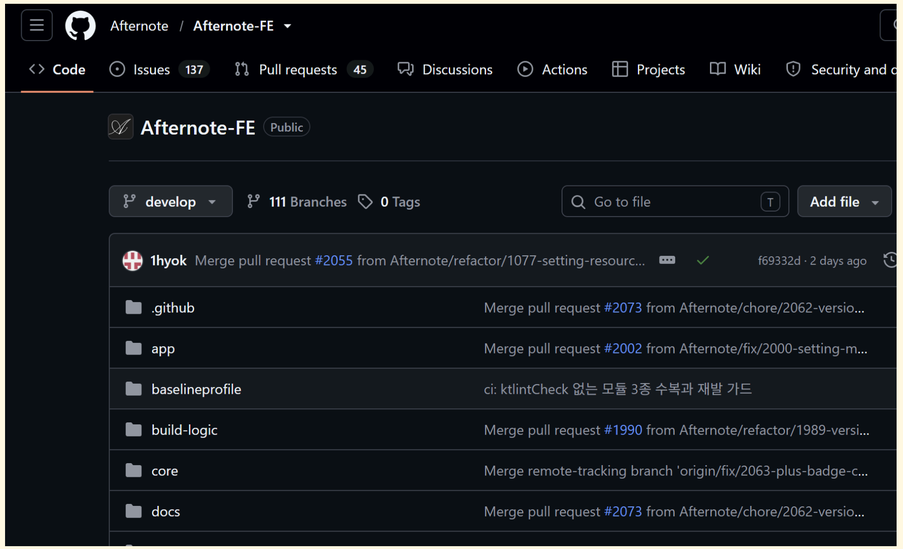
02 GITHUB 이란 ?

- Git으로 관리되는 프로젝트를 웹상에서 공유하고 협업할 수 있는 플랫폼
- 가장 널리 사용되는 Git 기반 호스팅 서비스

- 코드 저장 및 버전 관리 제공
- 이슈 관리(버그/요구사항) 제공
- 코드 리뷰 및 Pull Request 기능 제공
- 프로젝트 보드 및 문서화 기능 제공

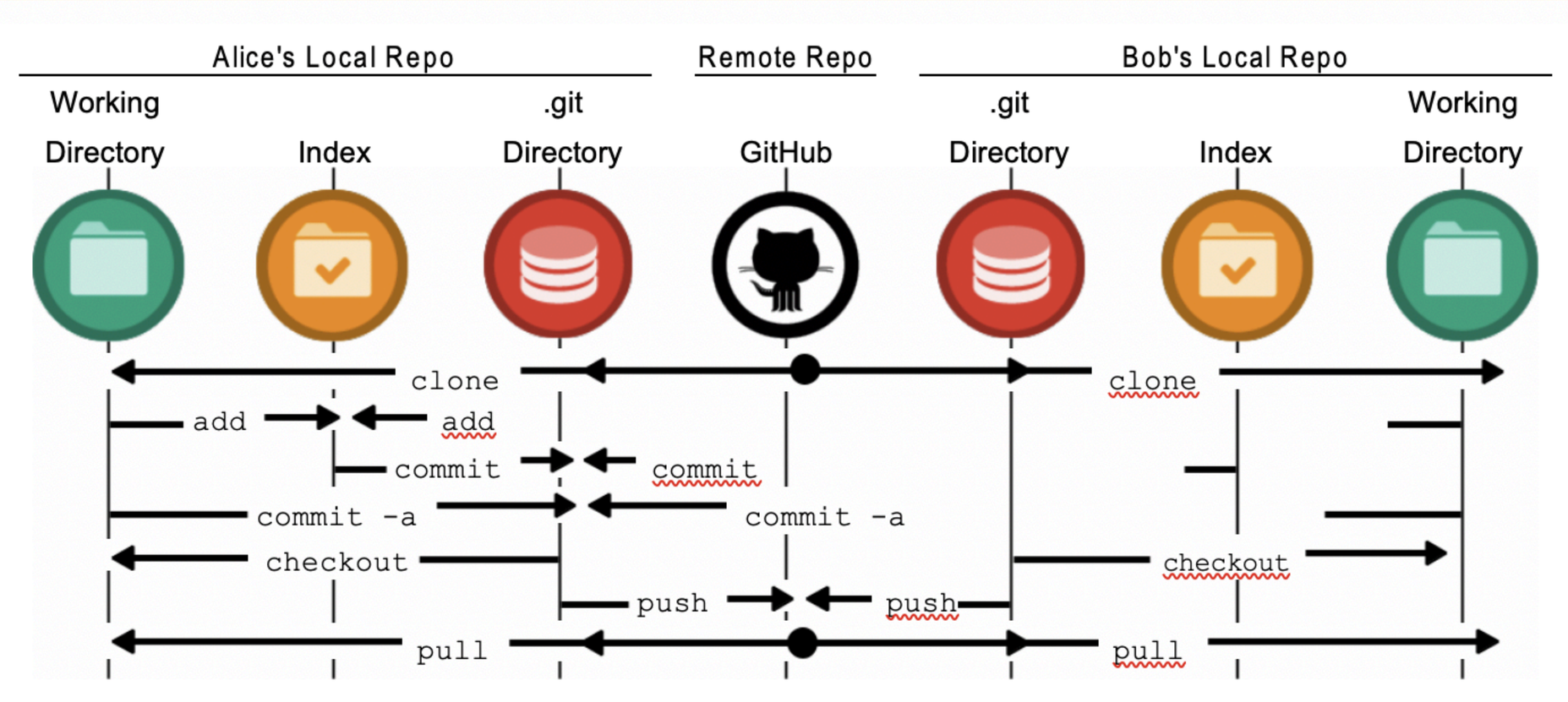
- 오픈소스 참여, 팀 프로젝트 협업, 개인 포트폴리오 관리에 활용
- 개발자 간 코드 공유, 수정, 협업 지원





01 GITHUB 이란 ?

GIT 과 GITHUB 차이

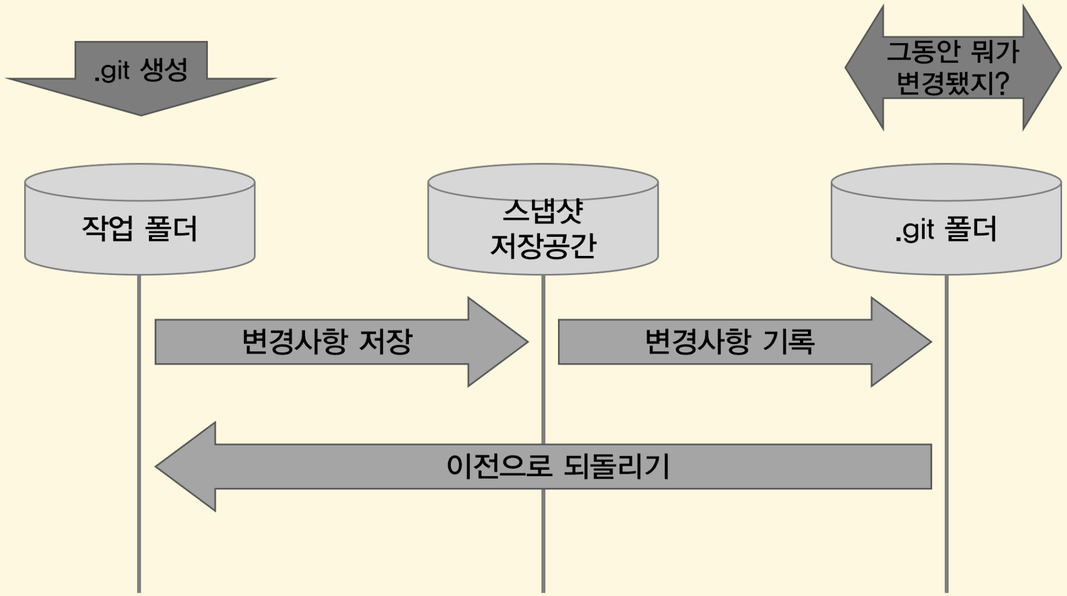


03

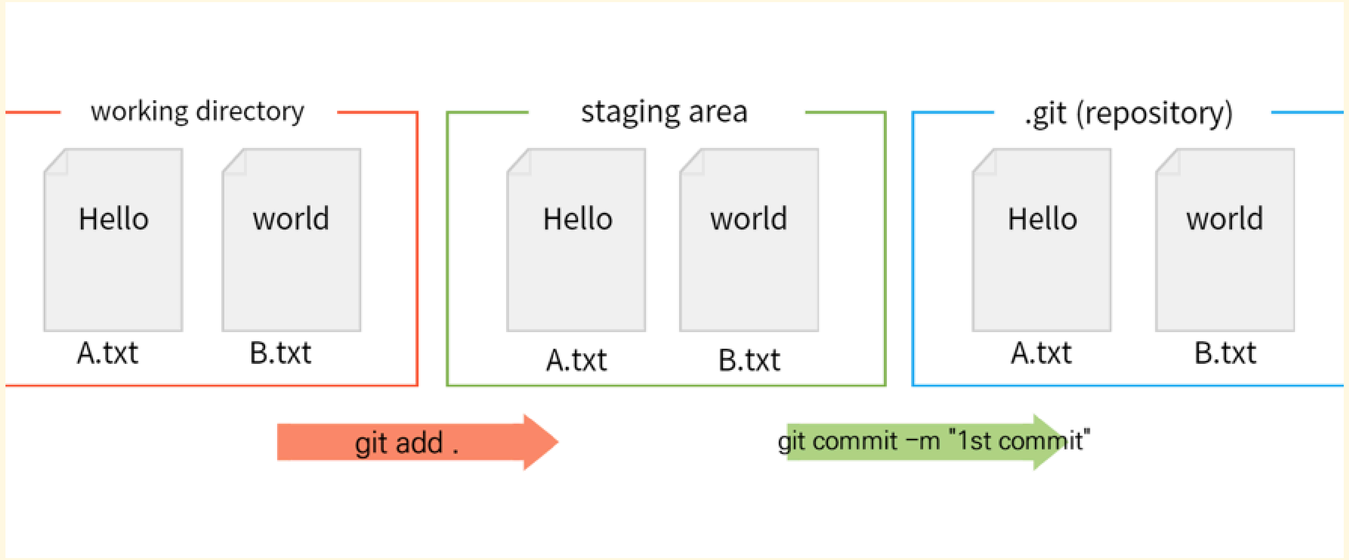
GIT의 3가지의 작업 영역

WorkingDirectory → Staging Area → Repository

3. GIT의 3가지 작업 영역

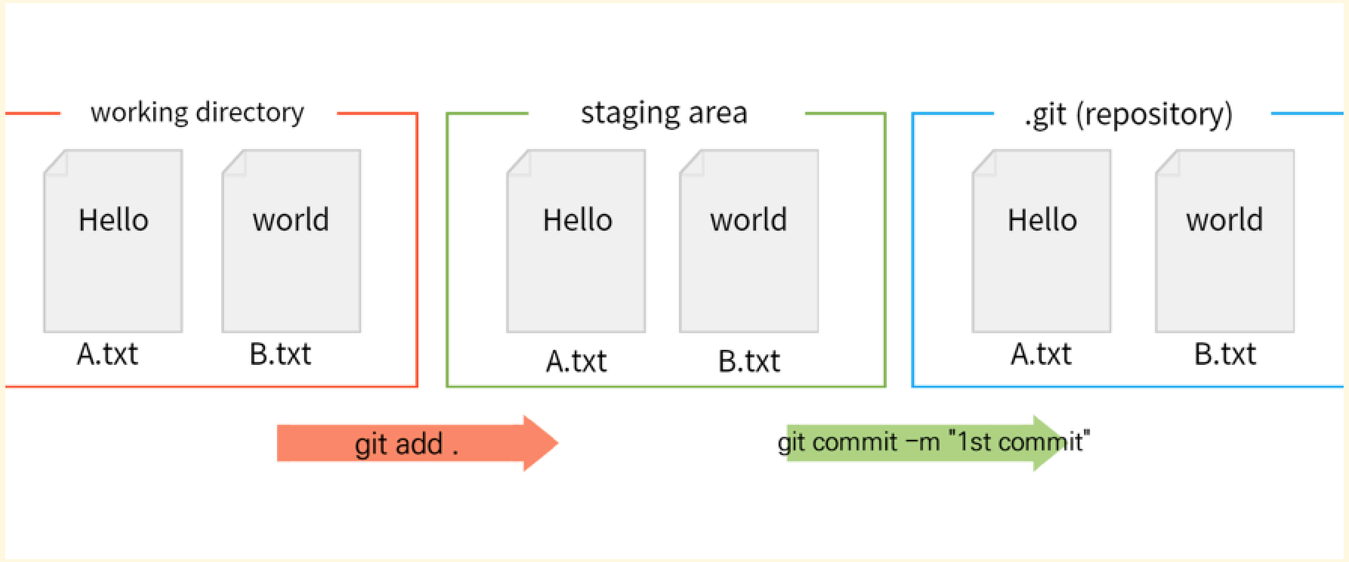


Copyright © 2024 All Rights reserved by yganalyst



working directory

working directory 란? 작업을 하는 공간
실제로 파일을 생성하고 수정하는 공간 ← 그냥 우리가 코딩하는 곳. 그냥 폴더



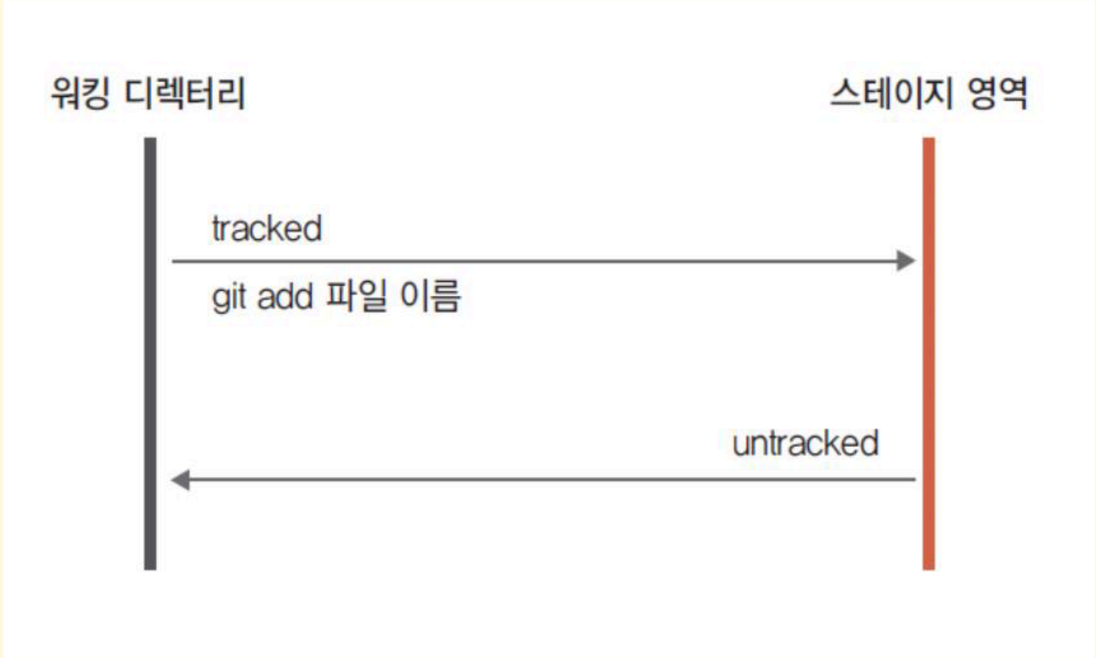
staging area

staging area 란 본래 저장소에 최종 반영하기 전,
변경된 데이터를 임시로 모아두고 검토하는 준비 단계의 공간

즉, 임시로 저장하는 공간 (git add 로 스테이징 시킨다.)

git add 가 하는 일

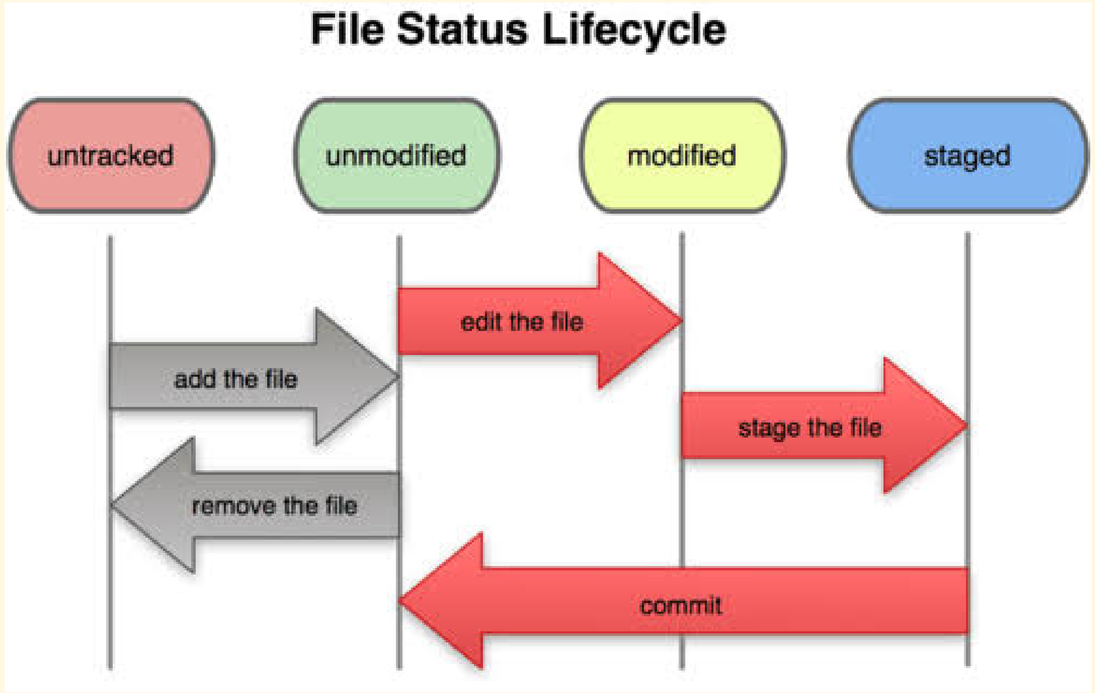
- untracked 파일에 add 하면 tracked 와 staged가 되고
- 한번 tracked 된 파일은 계속 tracked
- 근데 그 파일에 새로운 수정사항이 생기면 그 수정사항은 tracked 이지만 unstaging 인 상태
- git add 를 이용해서 다시 staged 해줘야함.



staging area

파일들의 스테이지 상태는git status명령어로 확인 할 수 있습니다.

워킹에서 자신이 저장한 파일들이 스테이지에 제대로 올라갔는지체크 해줘야됩니다.

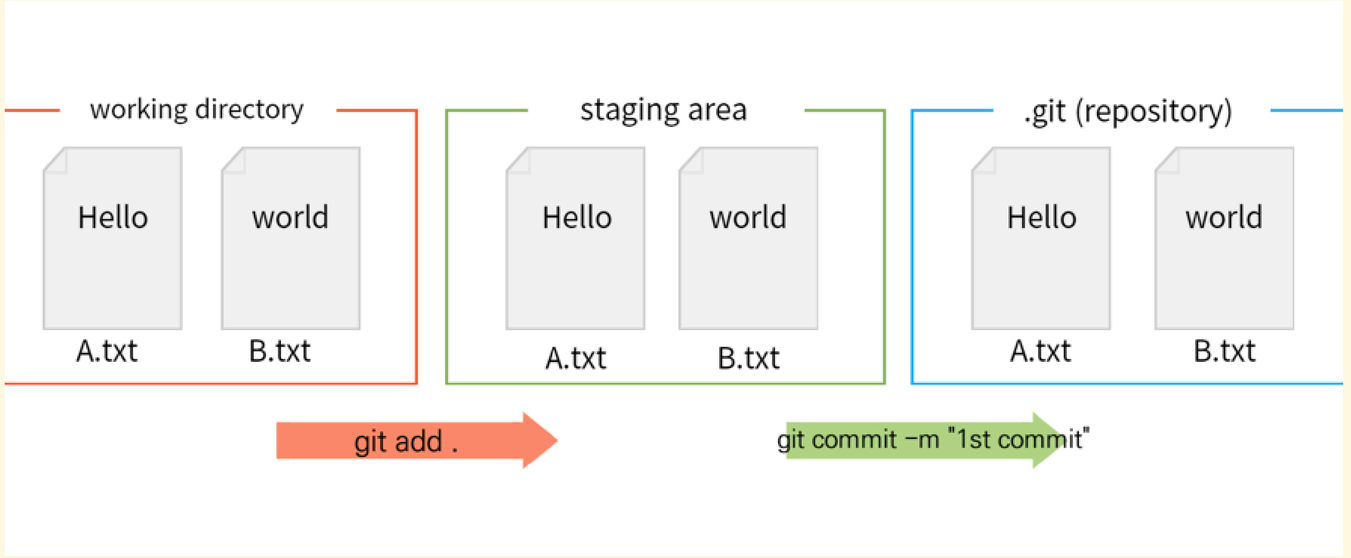


repository (로컬 레포지토리)

깃에서 작업한 이력을 저장하는 저장소 , 복귀지점

.git 에 버전관리 시스템에 필요한 파일 변경 이력을 기록

- .git 폴더 자체 (좁은 의미)
- 워킹 디렉토리 + .git 을 합친 프로젝트 폴더 전체 (넓은 의미)



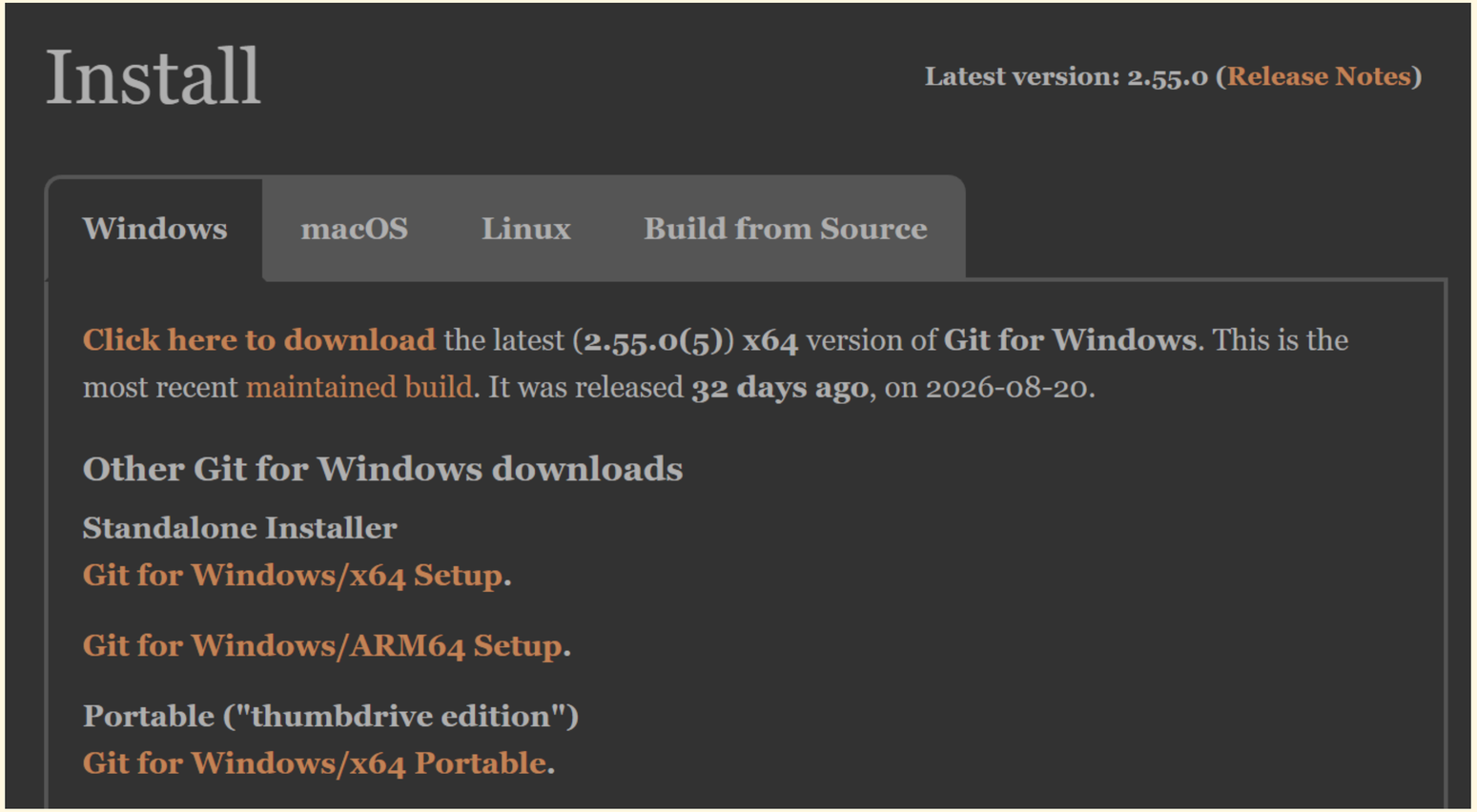
04

GIT 설치

Windows 전용

04 GIT 설치

1. https://git-scm.com/install/windows 링크 접속
2.



04 GIT 설치

3. next 를 계속 누르는데...

주의!!!!

- **기본 에디터 선택 (choosing default editor)**
  - vim 이 default 인데 visual studio code 선택 추천
- **기본 브랜치 이름 (Adjusting the name of the initial branch)**
  - “Override the default branch name for new repositories"를 체크하고 main으로 설정
- **줄바꿈 처리 방식 (Configuring the line ending conversions)**
  - “Checkout Windows-style, commit Unix-style line endings" (첫 번째 옵션)을 선택

04 GIT 설치

cmd 에 이렇게 잘 나오면 끝 & git bash 가 자동으로 설치되었을 텐데 잘 있는지 확인

```
C:\Users\박경민>git --version
git version 2.49.0.windows.1
```



05

실습 레츠고

커밋 하나 해봅시다

사용자 환경설정

사용자 정보를 .gitconfig 에 등록 해줘야 commit 을 할 수 있음

협업할때 " 이 코드 누가 작성/수정 했음?”이라고 따지기 위해서 이용한다.

파일 단위가 아니라 그 파일에 코드 한줄한줄 누가 어느시점에 수정했고 어떤줄을 어떻게 다르게 수정했고 등등을 전부 조회 가능하다.

git config --global user.email “깃헙 계정에 등록한 이메일”
git config --global user.name “자기이름 영어추천”

git config --list ← 이걸로 확인 가능

저장소를 생성하는 법

저장소를 생성하려면 초기화 작업을 해줘야 함.

여기서 말하는 초기화는 숨겨진 영역을 생성하는 것임.

.git 폴더를 만드는 작업이라고 생각해도 됨. git init 뒤에 경로명을 적으면 그 경로를 따라가서 초기화, 안 적으면 현재 폴더에서 초기화

git init

pwd ← 현재 워킹 디렉토리 출력
cd ‘원하는 경로’ ← 원하는 경로로 터미널 이동
mkdir 디렉토리이름 ← 디렉토리 만들기
cd 디렉토리이름 ← 여기로 이동
git init ← 현재 폴더에 .git 폴더를 만들어서
이 폴더를 Git 저장소(repository)로 만듦

확인은 어떻게? ls -a ← .git 이 있으면 잘 된 거.

우리의 워킹 디렉토리에서 아무 작업

touch hello.txt
code .

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be
committed)
        hello.txt

nothing added to commit but untracked files present (u
se "git add" to track)
```

```
박 경 민 @BOOK-V4BPCIJOV6 MINGW64 /d/bimbapgit (master)
$ git add hello.txt

박 경 민 @BOOK-V4BPCIJOV6 MINGW64 /d/bimbapgit (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt
```

로컬 저장소 (repository)에 저장 , commit

git commit -m “커밋 메세지”

```
박 경 민 @BOOK-V4BPCIJOV6 MINGW64 /d/bimbapgit (master)
$ git commit -m "하 이 하 이  추 가 "
[master (root-commit) 203fb35] 하 이 하 이  추 가
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

git commit -am “커밋 메세지”
← add 랑 commit 을 함께 할 수 있긴한데 따로따로 하는 습관추천

git log 로 커밋 내역 확인하기

```
박 경 민 @BOOK-V4BPCIJOV6 MINGW64 /d/bimbapgit (master)
$ git log
commit 203fb3589b5ed4224e3de25c82110414c675d035 (HEAD -> master)
Author: kyungmin <minii7582@konkuk.ac.kr>
Date:   Mon Sep 21 13:34:32 2026 +0900

    하 이 하 이  추 가
```

06 미션

미션 1. 커밋 여러 번 쌓기

미션 2. add 안 하고, commit 해보기

미션 3. 파일 두 개 선택 커밋하기

미션 4. git 폴더 들여다 보기